Final Project Phase 1 Report

Julian Canales
CS 378 Geometric Foundations of Data Science
Spring 2025
April 20, 2025

1 Introduction

Autonomous driving requires robust decision-making in the face of uncertainty. This report analyzes approaches for handling uncertainty in autonomous driving decision-making, focusing on the work by Leurent et al. [1]. The report is structured as follows:

1. Item A: Understanding the base paper and environment
2. Item B: Review of further methods explored
3. Item C: Proposed extension and experimental plan

2 Item A: Understanding the base paper and environment

2.1 Problem Formulation and Motivation

Autonomous vehicles operate in dynamic, uncertain environments where both perception systems and other traffic participants introduce uncertainty. Traditional control methods used to simulate physical processes (such as those that follow physical laws) fail to perform well addressing this problem for that reason. Hence, robust control frameworks needed to be developed that explicitly account for uncertainty.

In order to understand such robust control frameworks, we need to start with Reinforcement Learning. Reinforcement Learning (RL) provides a mathematical framework for sequential decision-making under uncertainty. In RL, an agent interacts with an environment modeled as a Markov Decision Process (MDP), which consists of:

- A state space S representing all possible configurations of an agent in an environment
- An action space A representing possible decisions an agent can make
- A transition function $T(s'|s,a)$ describing how the environment evolves from one state s to another state s' given some action a was performed
- A reward function $r(s,a)$ providing feedback on performance when the agent is in state s and performs action a

The goal is to find a policy $\pi(a|s)$ that maps states to actions and maximizes the expected cumulative reward. This is formally defined as finding a policy that maximizes:

$$R_\pi^T(s) \stackrel{\text{def}}{=} \sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \tag{1}$$

$$v_\pi^T(s) \stackrel{\text{def}}{=} \mathbb{E}(R_\pi^T(s)) \tag{2}$$

where $s_0 = s$ is the initial state, $a_t \sim \pi(s_t)$ is the action chosen by policy π at state s_t, $s_{t+1} \sim T(s_{t+1}|s_t, a_t)$ is the next state determined by the transition function, and $\gamma \in [0,1)$ is a discount factor that prioritizes near-term rewards.

Model-based RL methods attempt to learn an approximate model $\hat{T}$ of the environment dynamics to simulate experiences and optimize:

$$\max_{\pi} v_{\pi}^{\hat{T}}. \tag{3}$$

The limitation of this approach as stated in the base paper is model bias - the discrepancy between the learned model $\hat{T}$ and the true environment dynamics T. Schneider (1997) demonstrated that even small errors in the model can lead to dramatically degraded performance when policies are deployed in the real world. This is even more problematic in safety-critical applications like autonomous driving, where errors can have detrimental consequences.

To address model bias, robust control frameworks consider a range of possible models rather than relying on a single model. This approach introduces the concept of an ambiguity set.

An ambiguity set Θ represents the range of uncertainty about the true environment dynamics. In autonomous driving, this could include uncertainty about:

- How aggressively other vehicles accelerate or brake
- What safety margins other drivers maintain
- How drivers make lane-change decisions
- How road conditions affect vehicle handling

Mathematically, the ambiguity set is defined as a collection of parametrized deterministic dynamical systems $s' = T_\theta(s, a)$ where unknown parameters θ lie in a compact set $\Theta \subset \mathbb{R}^p$.

The robust objective then becomes:

$$\max_{\pi} \min_{T \in \{T_\theta : \theta \in \Theta\}} v_{\pi}^{T} \tag{4}$$

This formulation aims to find a policy that maximizes performance in the worst-case scenario across all plausible models, providing safety guarantees that are crucial for autonomous driving.

2.2 Algorithm Description

Leurent et al. propose two approaches to solve the robust control problem, each addressing different types of uncertainty structures.

1. Sampling-based Planning (Algorithm 1): This approach handles discrete uncertainty sets by constructing and exploring a look-ahead tree. The algorithm works as follows:

- **Tree Construction:** The algorithm builds a tree $\mathcal{T}$ where each node represents a joint state across all possible dynamics models $\{T_m\}_{m \in [1,M]}$. Starting from the current state as the root, each branch represents an action $a_k \in A$.
- **Value Computation:** For each node i of depth d in the tree, the algorithm computes three key values (these are taken directly from [1]):

- **Robust value (v_i^r):** The theoretical best worst-case performance when starting with action sequence i, defined as:

$$v_i^r = \max_{\pi \in iA^\infty} \min_{m \in [1,M]} R_\pi^{T_m} \tag{5}$$

  where iA^∞ is the set of policies that start with sequence i.

- **Robust u-value (u_i^r):** For leaf nodes, this is the worst-case discounted sum of rewards collected from the root to node i. It is then backed up to internal nodes:

$$u_i^r(n) = \begin{cases} \min_{m \in [1,M]} \sum_{t=0}^{d-1} \gamma^t r_t & \text{if } i \in L_n \\ \max_{a \in A} u_{ia}^r(n) & \text{if } i \in T_n \setminus L_n \end{cases} \tag{6}$$

- **Robust b-value (b_i^r):** An optimistic upper bound that guides exploration:

$$b_i^r(n) = \begin{cases} u_i^r(n) + \frac{\gamma^d}{1-\gamma} & \text{if } i \in L_n \\ \max_{a \in A} b_{ia}^r(n) & \text{if } i \in T_n \setminus L_n \end{cases} \tag{7}$$

- **Node Expansion:** The algorithm iteratively expands the leaf node with the highest b-value, as this node has the most potential to improve the policy.
- **Action Selection:** After exhausting the computational budget, the algorithm returns the action with the highest u-value, which represents the action with the best guaranteed worst-case performance.

An insight of this algorithm is that it takes the minimum over dynamics models only at leaf nodes, not at each step. This is meant to handle the non-rectangular structure of the problem, where the worst-case dynamics depends on the entire policy.

2. Interval Predictors (Algorithm 2): For continuous ambiguity sets where enumeration is impossible, this algorithm uses interval prediction to bound the possible states that can be reached:

- **Reachability Sets:** For a given policy π, initial state s_0, and time t, the reachability set $S(t, s_0, \pi)$ represents all possible states that could be reached:

$$S(t, s_0, \pi) = \{s_t : \exists \theta \in \Theta \text{ s.t. } s_{k+1} = T_\theta(s_k, a_k), a_k = \pi(s_k), k = 0, \ldots, t-1\} \tag{8}$$

- **Interval Hull Approximation:** Since these reachability sets have complex shapes, the algorithm approximates them by their interval hulls $\boxed{S}(t, s_0, \pi) = [\underline{s}(t, s_0, \pi), \overline{s}(t, s_0, \pi)]$, which are the smallest rectangular boxes containing all possible states.
- **Surrogate Objective:** The algorithm defines a surrogate objective over a finite horizon H:

$$\hat{v}^r(\pi) = \sum_{t=0}^{H} \gamma^t \min_{s \in \boxed{S}(t, s_0, \pi)} r(s, \pi(s)) \tag{9}$$

- **Policy Evaluation and Search:** The algorithm initializes a set of policies, evaluates each policy's surrogate objective, and updates the policy set through techniques like evolutionary strategies or other derivative-free optimization methods.

This approach provides a computationally tractable method for handling continuous ambiguity sets while maintaining safety guarantees.

2.3 Theoretical Guarantees

Both algorithms come with theoretical guarantees that ensure their effectiveness in finding robust policies:

Algorithm 1 (Sampling-based Planning): The performance of this algorithm is characterized by the simple regret $R_n = v^r - v^r_a$, which measures the difference between the optimal robust value and the value of the action a returned after n iterations.

The regret bound depends on a parameter $\kappa = K\gamma^\beta$, which captures the structure of the problem:

- K is the branching factor (number of possible actions)
- γ is the discount factor
- $\beta \in [0, \frac{\log K}{\log 1/\gamma}]$ characterizes the proportion of near-optimal nodes at each depth in the tree

The regret bounds are:

- If $\kappa > 1$ (harder problems with many near-optimal paths):

$$R_n = O\left(n^{-\frac{\log 1/\gamma}{\log \kappa}}\right) \tag{10}$$

  This shows that the algorithm converges to the optimal robust policy at a rate determined by the problem's complexity.

- If $\kappa = 1$ (easier problems with few near-optimal paths):

$$R_n = O\left(\gamma^{\frac{(1-\gamma)^\beta}{c}n}\right) \tag{11}$$

  Here, convergence is exponential in the number of iterations n.

These bounds guarantee that as computational resources increase, Algorithm 1 will find increasingly better approximations of the optimal robust policy, eventually converging to it.

Algorithm 2 (Interval Predictors): The key theoretical guarantee for this algorithm is that the surrogate objective $\hat{v}^r(\pi)$ is a provable lower bound on the true robust objective:

$$\forall \pi, \hat{v}^r(\pi) \leq v^r(\pi) \tag{12}$$

This property ensures that maximizing the surrogate objective will improve (or at least maintain) the true robust performance. The proof relies on the fact that minimizing rewards over the interval hull $\boxed{S}(t, s_0, \pi)$ will always yield a value less than or equal to the minimum over the true reachability set $S(t, s_0, \pi)$.

While this approach introduces some conservatism due to the interval approximation, it provides a guaranteed safety margin, which is crucial for autonomous driving applications where safety is paramount.

2.4 Environment Setup

The highway-env environment implements:

- **State:** Vehicle positions (x, y), velocities (v), and headings (ψ)
- **Action:** Discrete decisions {no-op, right-lane, left-lane, faster, slower}
- **Policy:** Maps vehicle states to tactical decisions
- **Reward:** Encourages fast driving while avoiding collisions

2.5 Experimental Results

Two scenarios were tested:

- **Discrete uncertainty:** Route choices at intersections
- **Continuous uncertainty:** Behavioral parameter variations

Key findings:

- Both robust approaches significantly outperformed the nominal planner in worst-case scenarios
- Algorithm 1: 8.99 vs. 2.09 worst-case return
- Algorithm 2: 7.88 vs. 1.99 worst-case return
- Both maintained good average performance comparable to an oracle with perfect knowledge

2.6 Advantages and Disadvantages

Advantages:

- Provides safety guarantees for worst-case scenarios
- Handles both discrete and continuous uncertainty
- Maintains good average performance
- Has theoretical performance guarantees

Disadvantages:

- High computational complexity
- Interval-based approach introduces conservatism
- Limited to discrete action spaces
- Potentially overly pessimistic

3 Item B: Review of further methods explored

Table 1: Comparison of Methods for Robust Autonomous Driving

Method	Model-based	Advantages and Limitations
Leurent et al. (Sampling) [Explored]	Yes	**Advantages:** Provides theoretical guarantees, handles discrete uncertainty sets. **Limitations:** High computational cost, limited to discrete action spaces.
Leurent et al. (Interval) [Explored]	Yes	**Advantages:** Handles continuous uncertainty, computationally efficient. **Limitations:** Conservative approximation due to interval overapproximation.
Adaptive Interval-based Robust Control [Shortlisted]	Yes	**Advantages:** Reduces conservatism of interval method. **Limitations:** Requires sufficient exploration data, additional implementation.
Cross-Entropy Method [Shortlisted]	Yes	**Advantages:** Simple sampling-based planning approach, already partially implemented. **Limitations:** Sample inefficiency, high computational cost for long planning horizons.
Bayesian Deep Learning [Rejected]	No	**Advantages:** Quantifies uncertainty in neural network predictions. **Limitations:** Computationally expensive, requires significant modifications.
Kalman Filter State Estimation [Rejected]	Yes	**Advantages:** Optimal state estimation under Gaussian assumptions, explicit uncertainty representation. **Limitations:** Assumes Gaussian noise, requires additional implementation.
Robust Model Predictive Control [Rejected]	Yes	**Advantages:** Handles constraints explicitly, can incorporate uncertainty. **Limitations:** Requires solving optimization problems online, complex implementation.

4 Item C: Proposed Extension and Experimental Plan

4.1 Proposed Extension: Adaptive Interval-based Robust Control

I propose an extension that enhances the interval-based robust control method of Leurent et al. by integrating an adaptive interval sizing mechanism grounded in Bayesian uncertainty estimation. This approach builds upon concepts from stochastic machine learning, Bayesian deep learning, and optimal control covered in our course. The method attempted is further informed by recent advances in adaptive robust control frameworks that utilize neural networks for uncertainty quantification in the references section.

The core innovation of this approach lies in its ability to dynamically calibrate interval widths through systematic analysis of observed prediction errors during runtime. Unlike fixed interval methods that apply uniform worst-case bounds across all states, our adaptive technique modulates conservatism based on empirical uncertainty, addressing a fundamental limitation of traditional robust control methods. This dynamic adjustment mechanism aligns with the Bayesian inference principles and uncertainty estimation techniques studied in class.

Building on Leurent's foundational work on interval predictive control [1] and incorporating elements from adaptive reinforcement learning algorithms [2], my extension consists of three interconnected components:

1. **Online Error Estimation Component:** Systematically track and analyze the empirical error distribution across different regions of the state space during execution, applying statistical machine learning principles to construct region-specific uncertainty models.
2. **Adaptive Interval Sizing Mechanism:** Dynamically adjust interval widths by applying confidence bounds derived from observed prediction errors, replacing static worst-case bounds with data-driven uncertainty estimates that become increasingly refined through experience.
3. **Confidence-Based Decision Framework:** Implement a probabilistic decision-making process that adaptively adjusts planning horizons and action selection strategies based on quantified uncertainty levels, integrating concepts from stochastic decision making.

4.2 Problem Formulation Using Adaptive Interval-based Robust Control

We formulate the robust autonomous driving problem as follows:

- **State space:** $s \in \mathcal{S}$ representing vehicle positions (x, y), velocities (v), and headings (ψ) for the ego vehicle and surrounding traffic participants.
- **Action space:** $a \in \mathcal{A}$ consisting of discrete decisions {LANE_LEFT, LANE_RIGHT, IDLE, FASTER, SLOWER}.
- **Uncertain dynamics:** We model the system as a parametrized family of deterministic dynamics $s_{t+1} = f_\theta(s_t, a_t)$ with unknown parameters $\theta \in \Theta$ representing

different driving behaviors.

The standard robust control objective aims to maximize the worst-case performance:

$$\max_{\pi} \min_{\theta \in \Theta} v_{\pi}^{f_\theta}(s_0) \tag{13}$$

where $v_{\pi}^{f_\theta}(s_0) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \mid s_0, \pi, f_\theta\right]$ is the expected return when following policy π under dynamics f_θ.

This approach extends the interval prediction method by incorporating online error estimation to dynamically adjust the bounds based on observed prediction errors.

4.3 Implementation Details

This section will provide structure of what my approach will look like through pseudocode. This can also be found in the attached demo where I ran the base code.

4.3.1 Online Error Estimation

```
class ErrorTracker:
    def __init__(self, state_size, buffer_size=1000, min_samples=50):
        # TODO: Initialize buffer to store (state, action, error) tuples
        # TODO: Set up parameters for error estimation
        pass

    def add_error(self, state, action, predicted_next, actual_next):
        # TODO: Calculate prediction error between actual and predicted
            states
        # TODO: Store the (state, action, error) tuple in buffer
        pass

    def get_error_bounds(self, state, action, confidence=0.95):
        # TODO: If insufficient samples, return default bounds
        # TODO: Find similar states/actions in buffer using similarity
            metric
        # TODO: Calculate statistical properties of errors for similar
            states
        # TODO: Compute confidence intervals based on statistical
            properties
        # TODO: Return lower and upper error bounds
        pass
```

4.3.2 Adaptive Interval-based Planning

```
class AdaptiveIntervalPredictor:
    def __init__(self, model, error_tracker):
        # TODO: Store reference to dynamics model and error tracker
        pass

```

```
    def predict_interval(self, state, action, confidence=0.95):
        # TODO: Get nominal next state prediction from model
        # TODO: Get adaptive error bounds from error tracker
        # TODO: Calculate interval bounds by adding errors to nominal
            prediction
        # TODO: Return lower and upper bounds for next state
        pass

    def update_statistics(self, state, action, next_state):
        # TODO: Get prediction from model for given state-action
        # TODO: Update error statistics with new observation
        pass
```

4.3.3 Integration with the CEM Planner

```
def adaptive_robust_cem_planner(x0, action_dim, model, error_tracker,
                            horizon=15, iterations=5, samples=500, topk
                                =50):
    # TODO: Initialize action distribution parameters (mean, std)
    # TODO: Create adaptive interval predictor using model and error
        tracker

    # CEM iterations
    # TODO: For each iteration:
    #   - Sample action sequences from distribution
    #   - Evaluate actions using adaptive intervals
    #   - Select top-performing sequences
    #   - Update distribution parameters based on top sequences

    # TODO: Return first action of best sequence
    pass

def evaluate_with_adaptive_intervals(x0, action_sequences, predictor):
    # TODO: For each action sequence:
    #   - Simulate trajectory using adaptive interval predictor
    #   - Calculate worst-case rewards over the interval
    #   - Return worst-case cumulative rewards for each sequence
    pass
```

4.4 Evaluation Metrics

We will evaluate this extension using:

- **Conservatism Reduction**: We will compare interval widths between our adaptive approach and fixed-interval methods, measuring the average reduction in interval size and how it varies across the state space.
- **Performance Gap**: We will measure the difference between worst-case and average performance to quantify how much performance is sacrificed for robustness.

- **Safety Rate**: We will track the percentage of episodes completed without collisions across varying levels of uncertainty.
- **Convergence Speed**: We will analyze how quickly interval widths stabilize in different regions of the state space as more data is collected.

4.5 Experimental Plan

The implementation and evaluation will proceed as follows:

```
# TODO: Setup experiment environment
def setup_environment():
    # Create highway-env environment
    # Configure observation and action spaces
    # Set up rendering for visualization
    pass

# TODO: Train baseline dynamics model
def train_dynamics_model(data):
    # Initialize neural network architecture
    # Train on collected state transition data
    # Validate model accuracy
    pass

# TODO: Implement error tracking module
def implement_error_tracker():
    # Create ErrorTracker instance
    # Set up data structures for storing observed errors
    # Implement similarity measures for state-action pairs
    pass

# TODO: Implement adaptive interval prediction
def implement_adaptive_predictor(model, error_tracker):
    # Create AdaptiveIntervalPredictor instance
    # Implement interval computation methods
    # Set up visualization for interval widths
    pass

# TODO: Evaluate approaches
def evaluate_approaches():
    # Test fixed-interval robust control
    # Test adaptive-interval robust control
    # Test non-robust baseline for comparison
    # Collect metrics on safety, performance, and interval widths
    pass

# TODO: Analyze results
def analyze_results(results):
    # Generate plots for conservatism reduction
```

```
    # Analyze performance across different uncertainty levels
    # Visualize interval adaptation over time
    # Compare safety statistics between approaches
    pass
```

5 References

1 Edouard Leurent, Yann Blanco, Denis Efimov, and Odalric-Ambrym Maillard. Approximate robust control of uncertain dynamical systems. arXiv preprint arXiv:1903.00220, 2019.

2 W. Guo et al., "Robust control for affine nonlinear system with unknown time-varying uncertainty under reinforcement learning framework," IET Control Theory & Applications, 2023.

3 X. Li et al., "Adaptive Interleaved Reinforcement Learning: Robust Stability of Affine Nonlinear Systems With Unknown Uncertainty," IEEE Transactions on Neural Networks and Learning Systems, 2020.

4 R. Sinha et al., "Adaptive Robust Model Predictive Control via Uncertainty Cancellation," arXiv preprint arXiv:2212.01371, 2022.